

✦ Member-only story

Teaching RAG to “Remember”: How MemoRAG Enhances Question-Answering Through Memory

Underlying Principles, Source Code, and Insights



Florian June · Following

Published in Towards AI

7 min read · 7 hours ago



Listen



Share



More

Existing RAG systems are limited in handling complex or ambiguous information needs that cannot be directly retrieved from external databases. For instance, traditional RAG systems excel in well-structured question-answering tasks but struggle when the task requires an implicit understanding of the underlying query or involves unstructured data retrieval.

A new study, “MemoRAG,” aims to address this issue by utilizing a long-term memory system that recalls relevant information based on context, significantly enhancing retrieval efficiency for complex tasks.

This article begins with an overview of MemoRAG, followed by **an explanation of its principles through code analysis**. Next, it details the training process and presents an evaluation along with a case study. **Finally, I will share my thoughts and insights on this approach, including a comparison with GraphRAG.**

Overview

As illustrated in Figure 1, Standard RAG, shown on the left, struggles to accurately locate the necessary evidence due to the implicit nature of the input query, resulting in a less precise answer. In contrast, MemoRAG, shown on the right, constructs a global memory across the entire database. When presented with the query,

MemoRAG first recalls relevant clues, enabling the retrieval of useful information and leading to a precise and comprehensive answer.

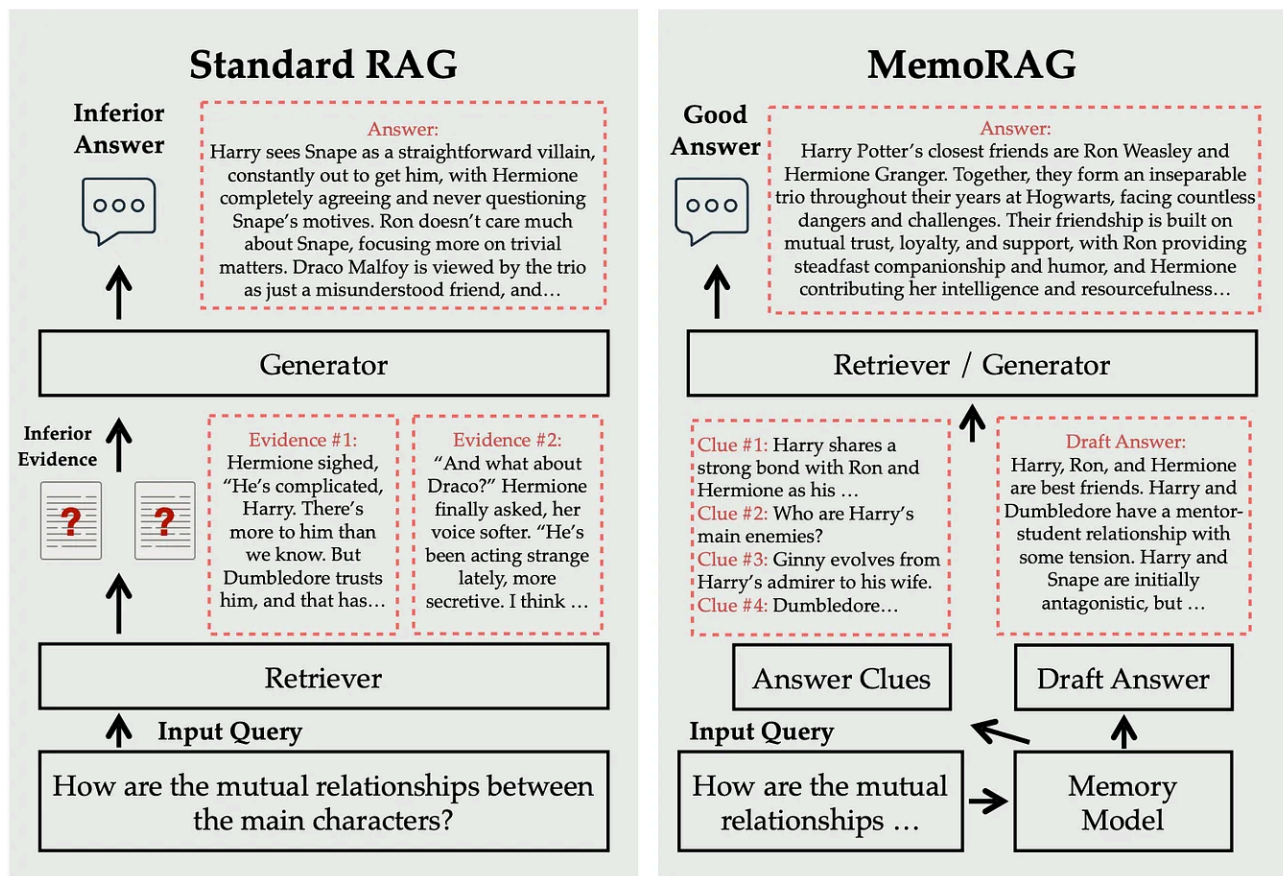


Figure 1: Comparison between Standard RAG and MemoRAG in processing queries that demand high-level understanding across the entire database, using the HARRY POTTER books as the database. Source: [MemoRAG](#).

Traditional RAG methods rely heavily on lexical and semantic matching, which can fall short when handling implicit information needs. MemoRAG, on the other hand, uses a lightweight but long-range LLM to build a global memory of the database, enabling it to recall key information and generate comprehensive responses. Its memory-based mechanism allows it to handle ambiguous queries and unstructured knowledge more effectively.

Detailed Mechanism

The underlying logic of MemoRAG is as follows: upon receiving a user query, MemoRAG first generates clues and potential(draft) answers based on memory. It then supplements these answers with detailed information from the retriever, ultimately producing a complete response. In other words, it unlocks hidden insights from complex data based on memory.

This section will provide a detailed analysis of the principles behind MemoRAG, using its [open-source code](#) as a reference. Let's first examine how to use MemoRAG.

The screenshot shows the GitHub repository for MemoRAG. The README is titled "Basic Usage of MemoRAG" and contains a code block with the following Python code:

```
from memorag import MemoRAG

# Initialize MemoRAG pipeline
pipe = MemoRAG(
    mem_model_name_or_path="TommyChien/memorag-mistral-7b-inst",
    ret_model_name_or_path="BAAI/bge-m3",
    gen_model_name_or_path="mistralai/Mistral-7B-Instruct-v0.2", # Optional: if not specify, u
    cache_dir="path_to_model_cache", # Optional: specify local model cache directory
    access_token="hugging_face_access_token", # Optional: Hugging Face access token
    beacon_ratio=4
)

context = open("examples/harry_potter.txt").read()
query = "How many times is the Chamber of Secrets opened in the book?"

# Memorize the context and save to cache
pipe.memorize(context, save_dir="cache/harry_potter/", print_stats=True)

# Generate response using the memorized context
res = pipe(context=context, query=query, task_type="memorag", max_new_tokens=256)
print(f"MemoRAG generated answer: \n{res}")
```

The code is annotated with three steps:

- Build and Store:** The first part of the code, from the import statement to the initialization of the `pipe` object, is highlighted in a blue box.
- Retrieve, and Generate:** The second part of the code, from the `pipe.memorize` call to the `print` statement, is highlighted in a blue box.
- Load:** The `pipe.load` call is highlighted in a blue box.

Below the code, there is a paragraph explaining the process:

When running the above code, the encoded key-value (KV) cache, Faiss index, and chunked passages are stored in the specified `save_dir`. Afterward, if the same context is used again, the data can be quickly loaded from the disk:

```
pipe.load("cache/harry_potter/", print_stats=True)
```

Typically, loading cached weights is highly efficient. For example, encoding, chunking, and indexing a 200K-token context takes approximately 35 seconds using `TommyChien/memorag-qwen2-7b-inst` as the memory model, but only 1.5 seconds when loading from cached files.

Figure 2: The basic steps of MemoRAG. Comment by author.

As shown in Figure 2, the entire process is divided into three steps: build and store index, retrieve, and generate responses. Next, we will explain these three steps.

Build and Store

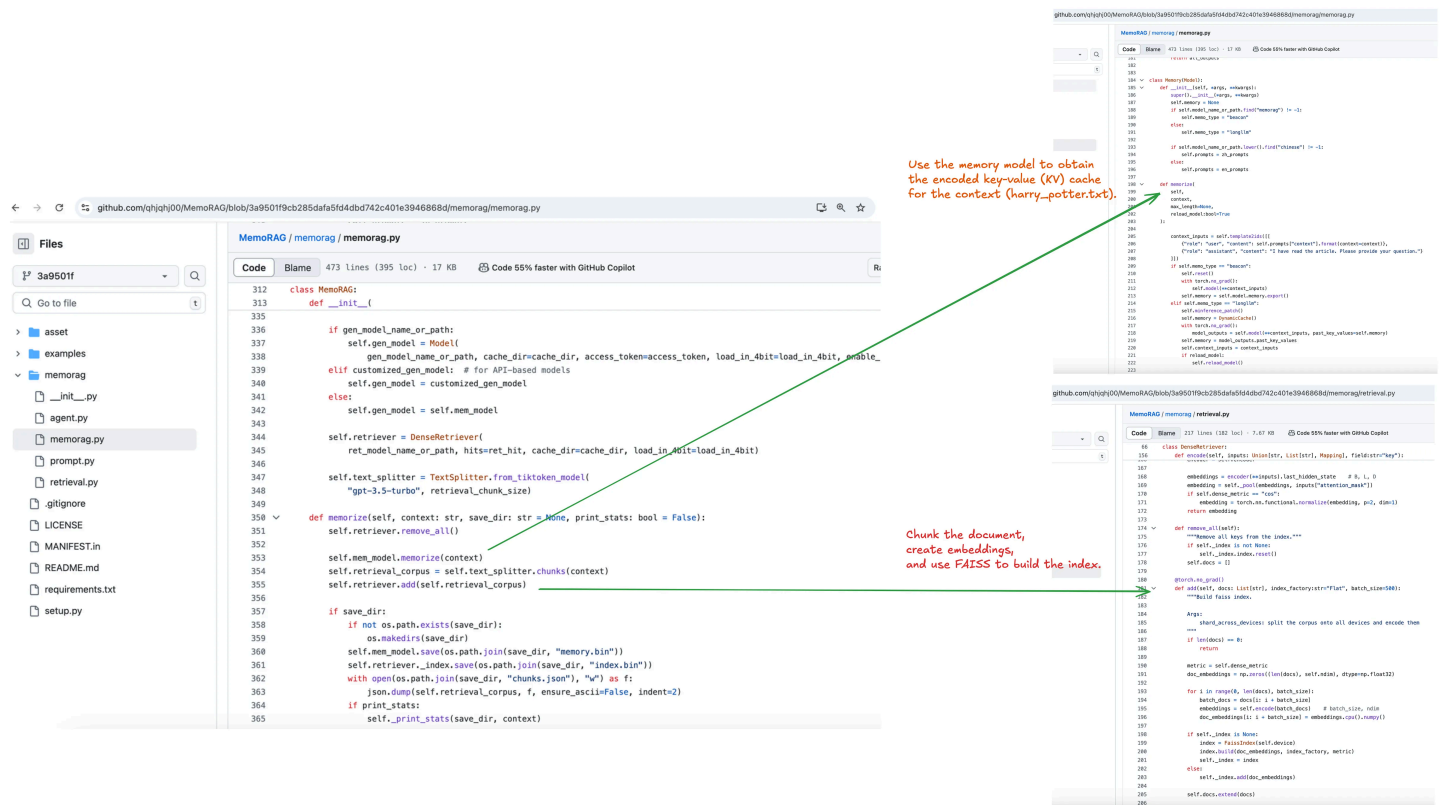


Figure 3: The main flow of building and storing. Comment by author.

The main flow of building and storing is shown in Figure 3. It consists of two steps:

- Use the memory model to obtain the encoded key-value (KV) cache for the context (e.g., `harry_potter.txt`).
- Chunk the original document, create embeddings, and use Faiss to build the index.

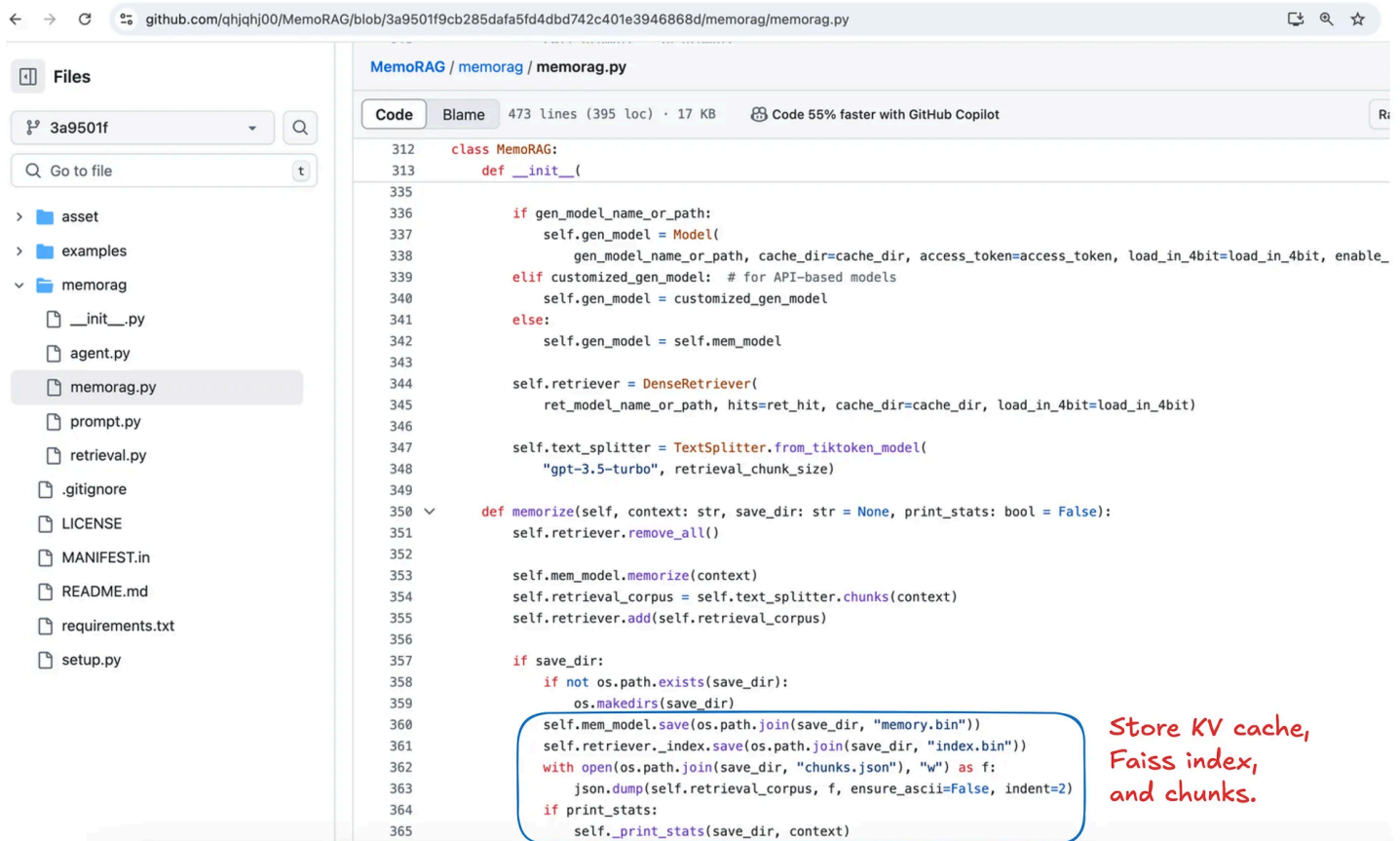


Figure 4: Store the created content. Comment by author.

Next, the previously created content should be stored in three key files:

- **memory.bin** : This file stores the key-value (KV) cache of the memory model, allowing for quick retrieval of previously processed information. It serves as a mechanism for rapid access to the knowledge stored in the model.
- **index.bin** : This file contains Faiss embeddings of the original document, facilitating the efficient retrieval of relevant paragraphs.
- **chunks.json** : This file includes the paragraphs or chunks extracted from the input context.

Retrieve and Generate Responses

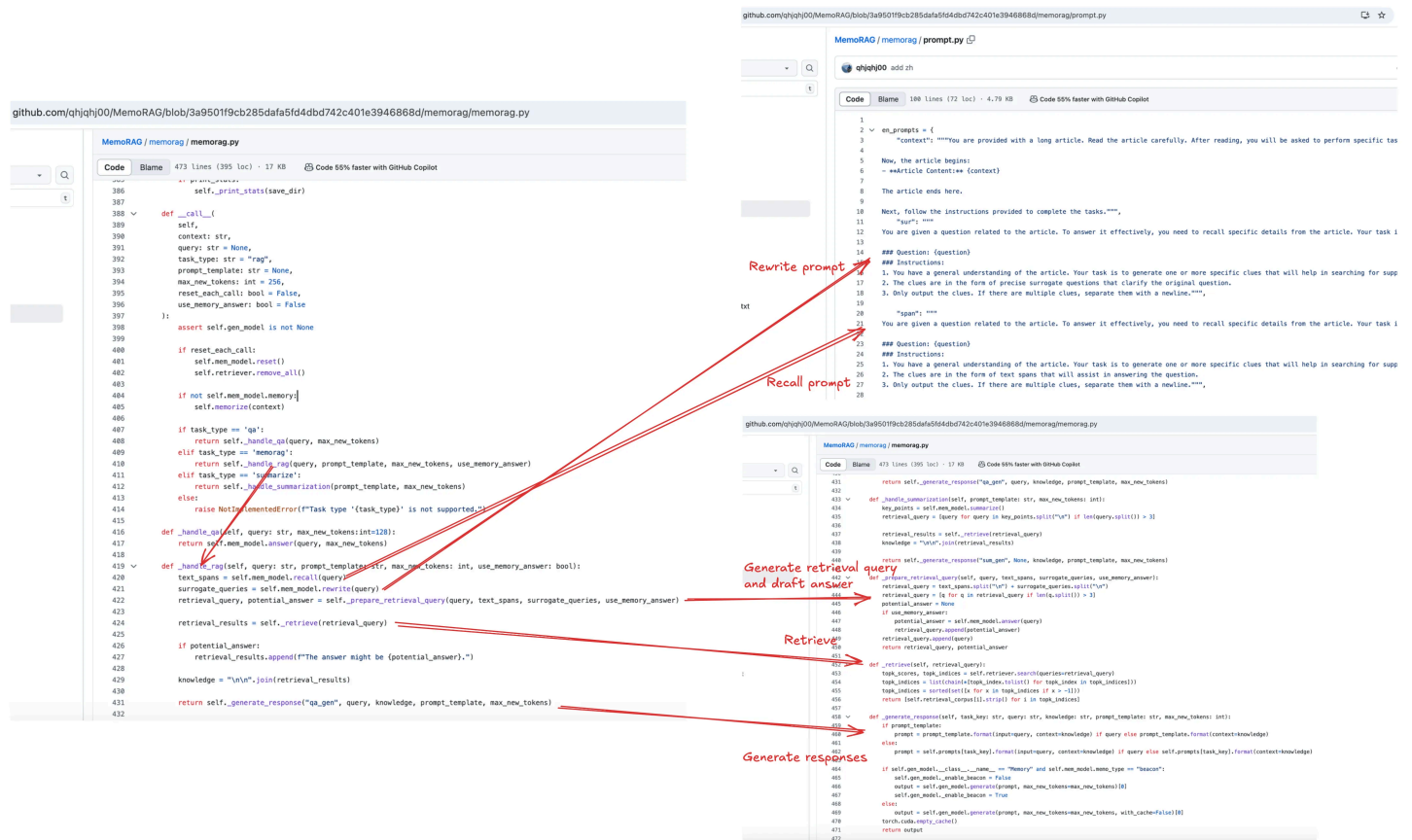


Figure 5: The process of retrieving and generating responses. Comment by author.

The main flow of retrieving and generating responses is illustrated in Figure 5, and it consists of six key steps:

1. Load the three key files through `pipe.load("cache/harry_potter/", print_stats=True)` , as shown in Figure 2.
2. Recall clue text using the memory model.
3. Rewrite the query using the memory model to generate a clue query.
4. Generate the retrieval query and potential(draft) answer by combining the clue text and clue query with length filtering.
5. Retrieve relevant chunks using the Faiss index.
6. Generate the response using the generation model.

Training the Memory Module

The memory module in MemoRAG is essential for effectively storing and recalling large amounts of information. Its training involves two key stages: pre-training and supervised fine-tuning (SFT).

1. Pre-training with Long Contexts

In the pre-training phase, the memory model learns to compress raw input into memory tokens using long-context data from the RedPajama dataset.

This process enables the model to retain important semantic information while discarding less relevant details. Essentially, it mimics how humans remember key points after reading long articles.

2. Supervised Fine-tuning (SFT)

After pre-training, the memory module undergoes supervised fine-tuning, where it is trained on labeled datasets containing queries and their corresponding answers.

This stage focuses on generating task-specific clues that guide information retrieval. For instance, in a financial analysis task, the model may generate clues like “revenue growth,” aiding the retrieval of relevant information.

The labeled datasets for SFT are constructed using the six steps illustrated in Figure 6.

Step	Description
1. Data Collection	Gather long contexts from diverse sources: novels, academic papers, news articles, financial reports, and legal contracts. Utilize existing datasets like NarrativeQA, Qasper, and HotpotQA.
2. Sampling Long Contexts	Sample long contexts with lengths of up to 80,000 tokens to ensure rich information for generating question-answer pairs.
3. Generating Question-Answer Pairs	Prompt advanced language models (e.g., GPT-4 128K, Deepseek-v2-128K) to create insightful question-answer pairs based on the sampled contexts.
4. Quality Inspection	Conduct quality inspection of the generated question-answer pairs and select 20,000 high-quality samples for further processing.
5. Generating Answer Clues	Use the selected samples to prompt the same language models to generate answer clues that connect queries to long contexts. Provide the model with the query, context, and answer.
6. Final Filtering	Inspect and strategically filter the generated answer clues to enhance quality, resulting in 17,116 final SFT training samples.

Figure 6: The construction steps of SFT datasets. Image by author.

The training objective maximizes the probability of generating the next token based on previous tokens and memory.

Evaluation

MemoRAG was evaluated using a comprehensive benchmark called ULTRADOMAIN, which includes tasks from diverse domains such as law, finance, and education.

ULTRADOMAIN	Full	BGE-M3	Stella-v5	HyDE	MemoRAG	ave(C) (K)
In-domain						
Legal	35.8	<u>42.0</u>	34.9	35.1	51.2 ↑9.2	51.4
Financial	36.5	<u>40.5</u>	40.9	<u>42.8</u>	48.0 ↑5.2	40.6
Mix	42.1	41.1	42.1	<u>43.9</u>	53.6 ↑9.7	20.3
Average	38.1	<u>41.2</u>	39.3	40.6	50.9 ↑9.7	37.4
Out-of-domain						
Biology	<u>34.1</u>	32.2	32.1	31.5	35.7 ↑1.6	125.2
Religion	<u>36.7</u>	35.2	34.1	34.7	37.8 ↑1.1	131.4
Computer	<u>36.5</u>	35.9	32.9	35.5	40.5 ↑4.0	215.9
Fiction	<u>29.0</u>	27.6	26.5	27.1	31.3 ↑2.3	137.7
Literature	<u>30.5</u>	29.6	28.8	29.2	34.4 ↑3.9	129.4
History	<u>33.3</u>	31.9	32.3	31.1	35.6 ↑2.3	195.2
Biography	<u>32.4</u>	31.1	29.8	30.3	35.3 ↑2.9	163.5
Physics	36.4	38.1	37.3	<u>38.2</u>	38.8 ↑0.6	105.8
Music	<u>33.9</u>	33.5	31.5	32.9	35.1 ↑1.2	168.7
Art	32.5	<u>33.7</u>	33.1	33.0	36.6 ↑2.9	129.0
Mathematics	34.5	35.0	33.8	<u>35.4</u>	36.4 ↑1.0	198.0
Health	<u>34.8</u>	33.2	32.9	31.9	37.4 ↑2.6	134.9
Psychology	<u>34.3</u>	33.6	31.9	33.0	37.6 ↑3.3	150.1
Technology	<u>33.9</u>	32.5	31.1	31.8	37.4 ↑3.5	144.0
Politics	<u>33.0</u>	32.5	30.2	32.1	35.2 ↑2.2	139.6
Cooking	<u>34.1</u>	33.1	31.0	32.9	35.6 ↑1.5	156.1
Agriculture	<u>34.9</u>	34.0	33.2	32.8	36.7 ↑1.8	151.0
Philosophy	<u>33.0</u>	32.5	31.8	32.2	36.2 ↑3.2	135.7
Average	<u>33.8</u>	33.0	31.9	32.5	36.2 ↑2.4	150.6

Figure 7: Main experiment results on ULTRADOMAIN using Mistral-7B-Instruct-v0.2–32K as the generator. The evaluation metric is the F1-score, with the best results highlighted in bold and the second-best results underlined. The upward arrow ↑ indicates the improvement over the second-best results. ave(|C|) refers to the average context length, counted in thousands of tokens (K). Source: [MemoRAG](#).

As shown in Figure 7, MemoRAG outperformed traditional RAG models, especially in handling tasks involving long input contexts and implicit information needs. For example, in legal and financial tasks, MemoRAG achieved significantly higher accuracy and precision compared to other models. This demonstrates the effectiveness of MemoRAG in both complex and straightforward question-answering tasks.

Case Study

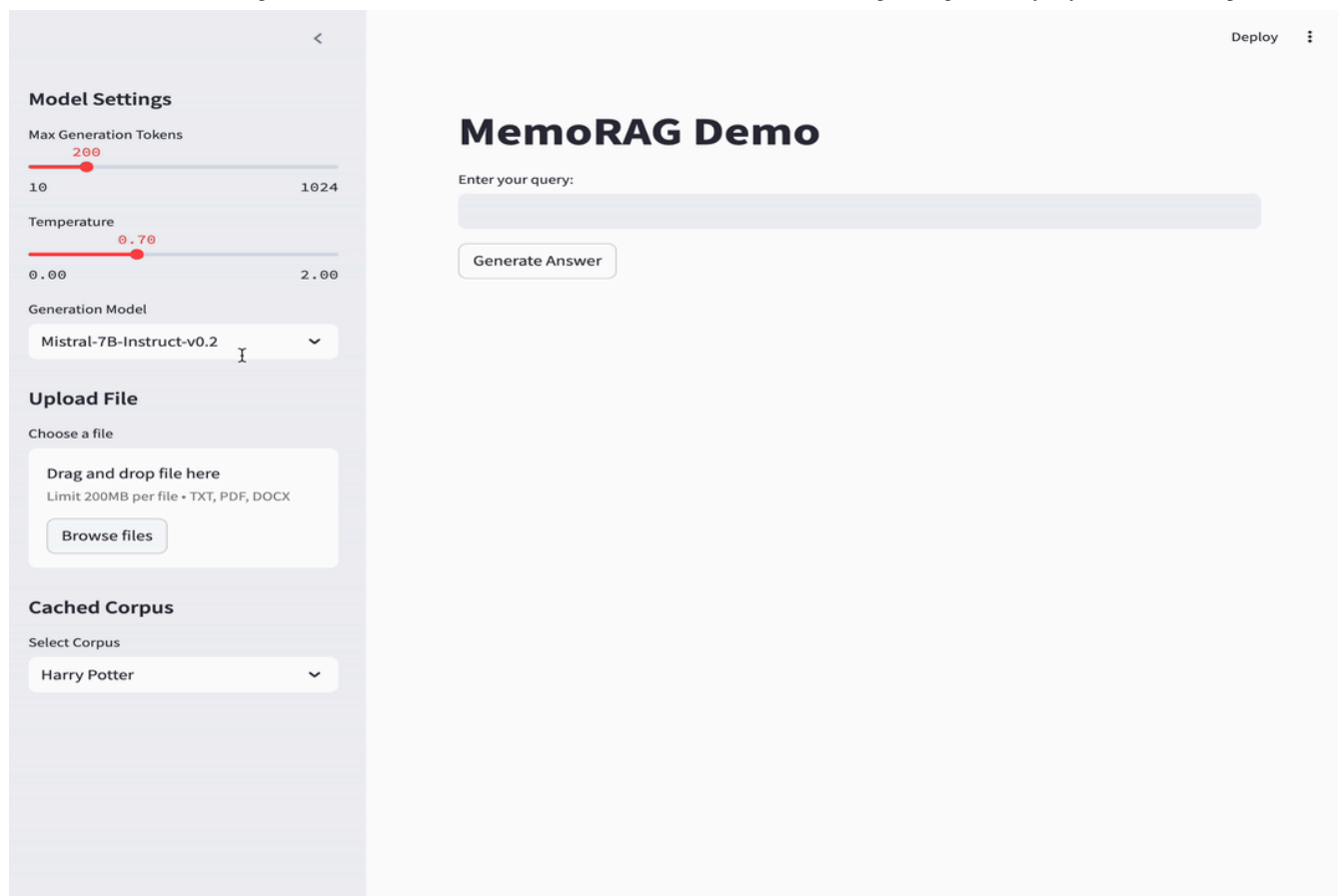


Figure 8: A case study of MemoRAG. Source: <https://github.com/qhjghj00/MemoRAG>.

MemoRAG provides a demo interface, which can be accessed by running `streamlit run demo/demo.py`.

First, load the corresponding corpus, index, and memory, then follow these three steps, as shown in Figure 8.

1. Generate clues.
2. Use the clues as evidence to retrieve the relevant passages.
3. Generate answers based on the retrieved passages.

My Thoughts and Insights

Innovations

In summary, MemoRAG’s innovations lie in its ability to:

- Form and use a global memory for long-context retrieval.
- Generate retrieval clues to improve information access for ambiguous queries.
- Significantly outperform standard RAG systems in tasks with long or complex inputs.

Challenges

However, I believe that challenges remain in MemoRAG.

- **Memory Accuracy:** Although the memory module can store essential information, ensuring the generated clues are both accurate and useful is a persistent challenge. This issue can affect the system’s overall performance when handling complex queries.
- **Real-World Application Challenges:** Implementing MemoRAG in real-world scenarios may encounter issues such as rapidly changing data and diverse information sources. The system must adapt quickly to new data to sustain its performance.

Comparison of MemoRAG and GraphRAG

To compare MemoRAG and GraphRAG, we evaluate their similarities and differences across key aspects such as architecture, target use cases, performance improvements, scalability, and application focus. Figure 8 summarizes the comparison between these two methods.

Aspect	MemoRAG	GraphRAG
Core Concept	MemoRAG introduces a memory-enhanced approach to improve retrieval-augmented generation (RAG) by leveraging long-term memory.	GraphRAG creates a graph-based text index for query-focused summarization using communities derived from text entities.
Architecture	Uses a dual-system architecture: a light long-range memory model to generate draft answers and a heavy LLM to refine them.	Utilizes a graph-based indexing pipeline with communities and summarization at both indexing and query time stages.
Memory System	Employs a long-term memory model to store and recall knowledge, which enhances retrieval by generating clues for ambiguous queries.	Builds an entity-relationship knowledge graph , which uses community detection for query summarization.
Target Use Case	Designed to handle complex RAG tasks where queries involve ambiguous or unstructured knowledge, like character relationships in books.	Focuses on query-focused summarization over large corpora, such as finding themes or answering global questions across datasets.
Handling Long Contexts	Supports handling long contexts using memory compression techniques , allowing for more efficient retrieval and generation over long input contexts.	Balances context size using chunking and community detection to extract and summarize relevant nodes and relationships.
Retrieval Approach	Combines memory-based retrieval clues with traditional RAG methods to retrieve and generate more accurate information.	Relies on graph-based retrieval and community summarization to retrieve and generate global answers to queries.
Performance Improvements	Claims superior performance in both simple and complex retrieval tasks by optimizing memory retention and clue generation.	Shows improvements in comprehensiveness and diversity of answers compared to a naive RAG baseline by using community summaries.
Application Domains	Proposed for multidomain tasks like finance, healthcare, law, and programming, where RAG systems traditionally struggle.	Aimed at dataset-wide summarization , such as for news articles, podcasts, and corpora requiring thematic analysis.

Figure 9: Comparison of MemoRAG and GraphRAG. Image by author.

In summary, while both MemoRAG and GraphRAG aim to enhance the RAG process by addressing limitations in handling long contexts and ambiguous queries, they approach the problem differently.

MemoRAG focuses on **long-term memory integration** to enhance retrieval quality, especially for complex, unstructured tasks, whereas GraphRAG emphasizes **graph-**

based indexing and query-focused summarization to handle large datasets with more diverse and comprehensive answers.

Conclusion

This article presented MemoRAG, an advanced RAG system that addresses many of the limitations faced by traditional RAG systems in complex tasks by introducing a memory module. It has great potential in both academic research and real-world applications, particularly in scenarios that require long-context and multi-hop reasoning. However, further exploration is needed to optimize resource consumption and improve the robustness and adaptability of the memory module.

Overall, MemoRAG represents a breakthrough in the RAG domain, **but like GraphRAG, it uses a space-for-performance trade-off as a specific trick.** The actual outcome remains to be seen.

Additionally, if you're interested in RAG, feel free to check out my other articles.

The image is a composite of three parts. On the left is a social media post from Florian June titled 'RAG' with a 'View list' button and '24 stories'. In the center is a diagram comparing 'Standard RAG' and 'MemoRAG'. 'Standard RAG' shows a linear flow from 'Generator' to 'Retriever' to 'Query'. 'MemoRAG' shows a more complex flow involving 'Retriever / Generator', 'Answer Clues', 'Draft', and 'Mer Mc'. On the right is a timeline titled 'In 1990', 'Marathon was renamed to S', 'In 1930', 'Snickers was created in', 'In 1991', with a 't=7' label at the bottom.

And the latest article or video can be found in [my newsletter](#) or on [YouTube](#).

Lastly, if there are any errors or omissions, or if you have any thoughts to share, please feel free to discuss in the comments section.

Large Language Models

Retrieval Augmented

ChatGPT

AI

Machine Learning



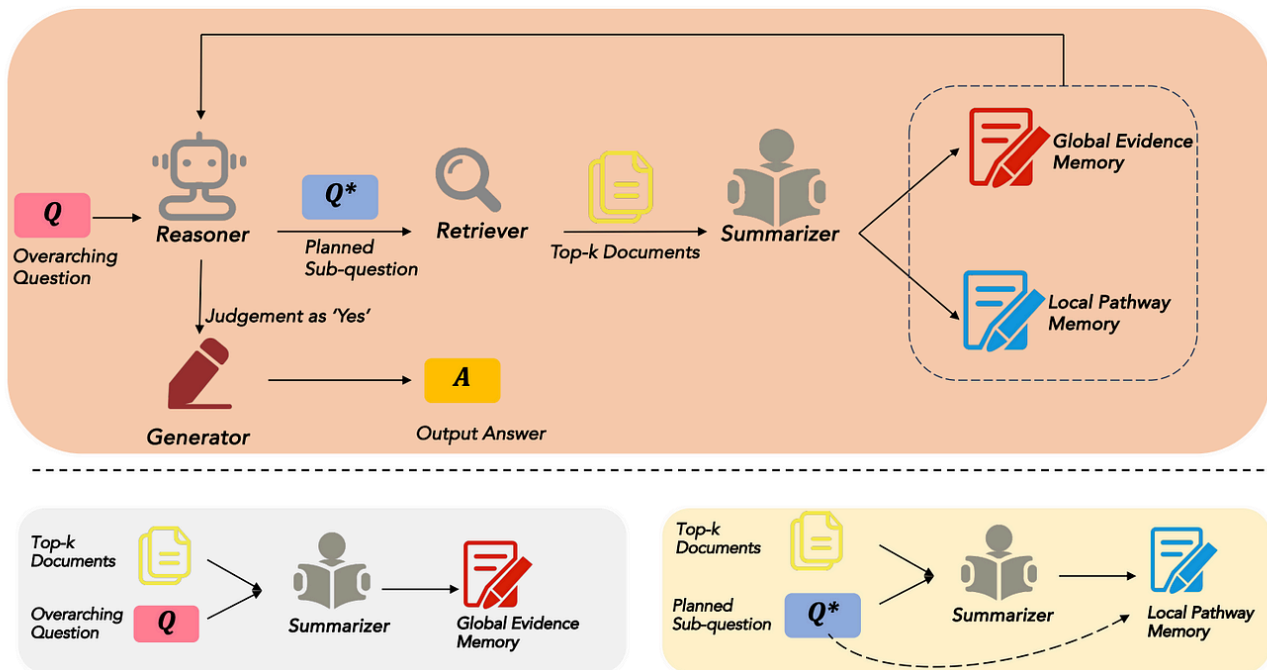
Following

Written by Florian June

8.4K Followers · Writer for Towards AI

AI researcher, focusing on LLMs, RAG, Agent, Document AI. Newest articles: florianjune.substack.com.Youtube: youtube.com/@ai_exploration_journey

More from Florian June and Towards AI



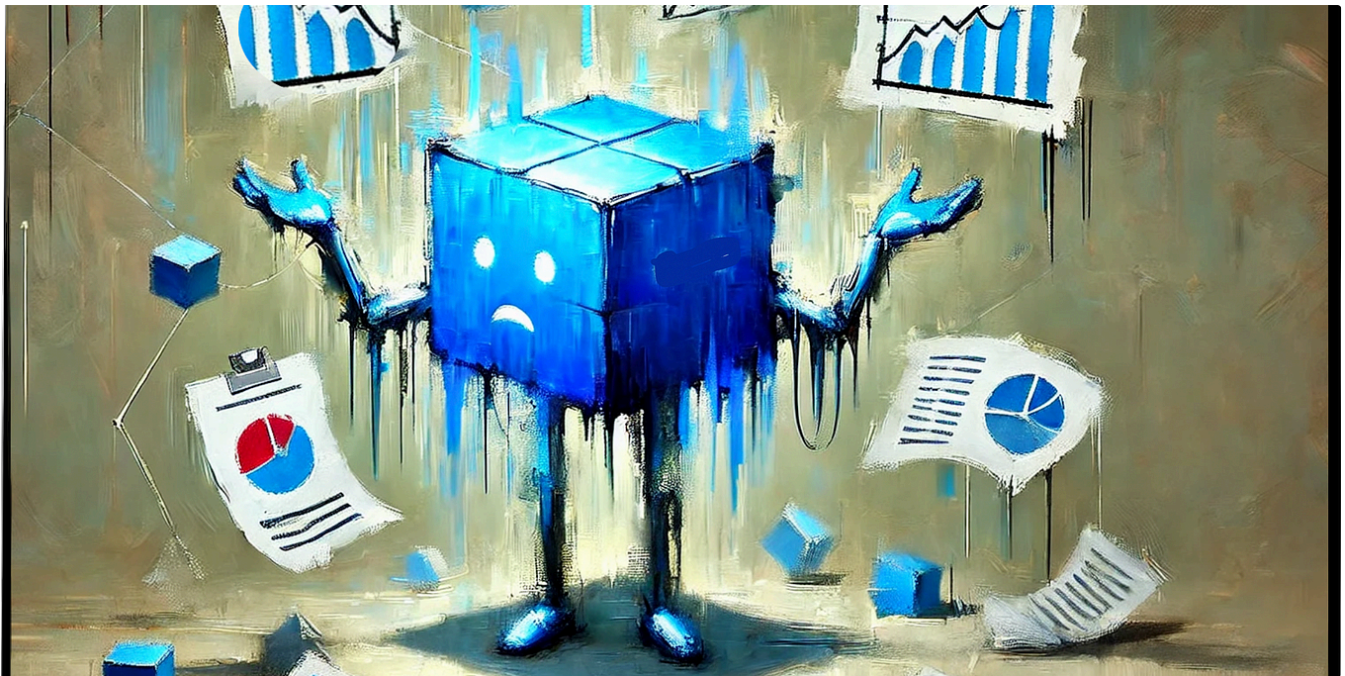
Florian June

An Innovative RAG Idea for Multi-hop Question Answering

In open-domain question answering, multi-hop question answering is complex and challenging. It requires the system to integrate information...

★ Sep 6 🤝 106



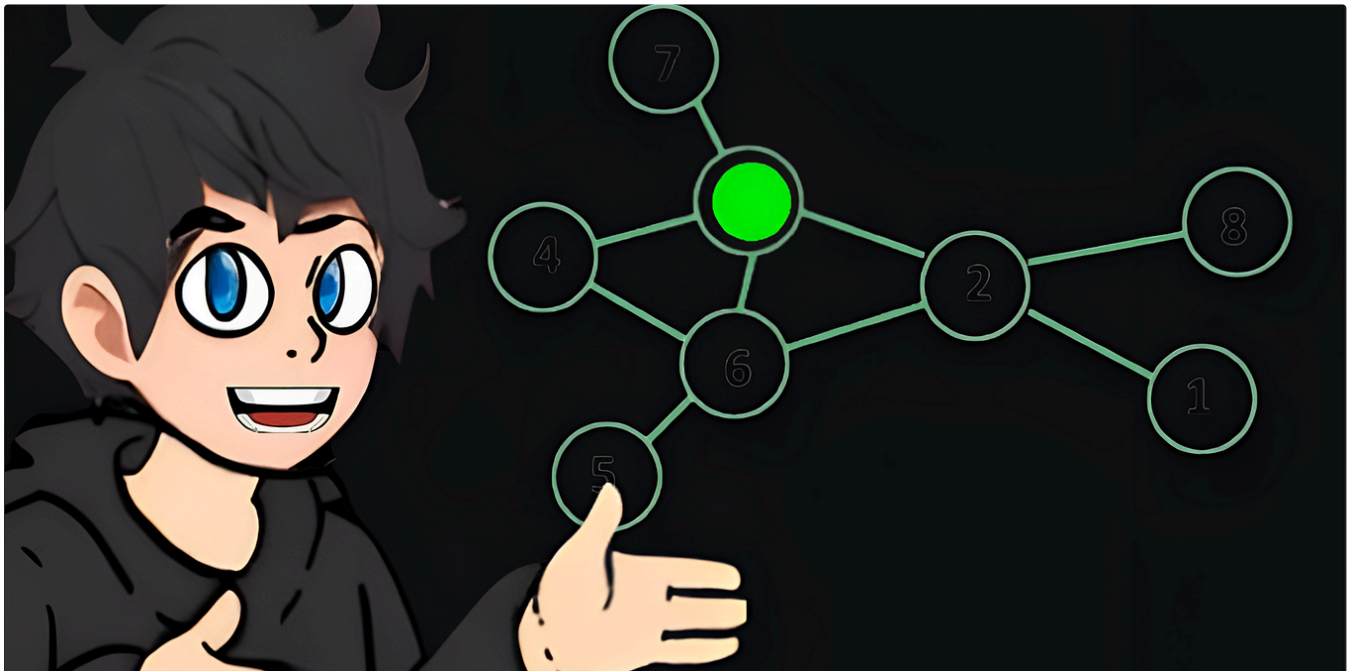


John Loewen, PhD in Towards AI

What Can GPT-4 Do Now With Data Visualization Report Generation?

The state of affairs with automatic data visualization reporting

★ Sep 12 🖱 311 💬 4

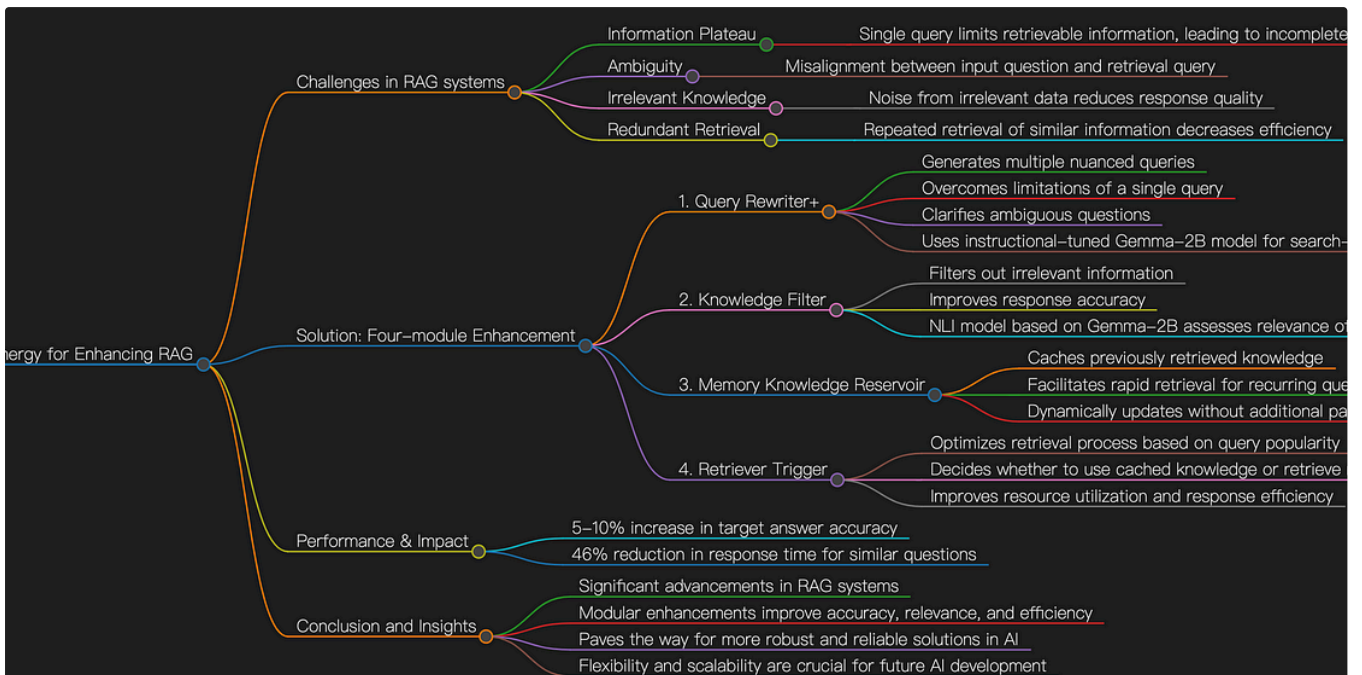


Gao Dalie (高達烈) in Towards AI

Langchain + Graph RAG + GPT-4o Python Project: Easy AI/Chat for your Website

This is Graph and I have a super quick tutorial showing how to create a fully local chatbot with Langchain, Graph RAG and GPT-4o to make a...

★ Jul 7 🖱 2K 💬 5



Florian June

Four-module Synergy for Enhancing RAG

Today, we'll explore a new development in RAG: the addition of effective modules aimed at solving several key challenges within RAG.

★ Sep 14 🖱 131 💬 3

[See all from Florian June](#)[See all from Towards AI](#)

Recommended from Medium



The evolution of language models has followed two major paths: generative models (such as GPT, Llama and T5) and discriminative models...



 Mahernaija

Large Language Models (LLMs) like GPT-4, BERT, and other transformer-based models have revolutionized the AI landscape. These models demand...

Aug 27

👏 69

💬 4



Lists



The New Chatbots: ChatGPT, Bard, and Beyond

12 stories · 463 saves



What is ChatGPT?

9 stories · 440 saves



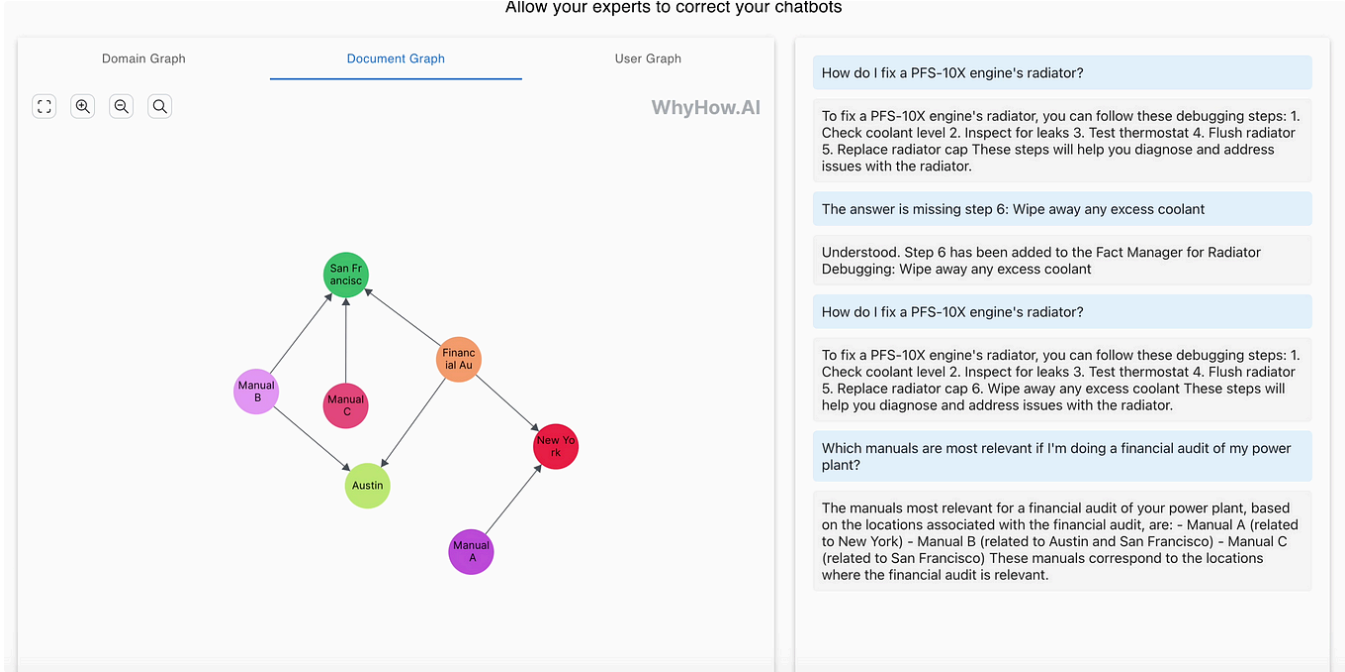
ChatGPT prompts

48 stories · 2027 saves



Natural Language Processing

1720 stories · 1296 saves

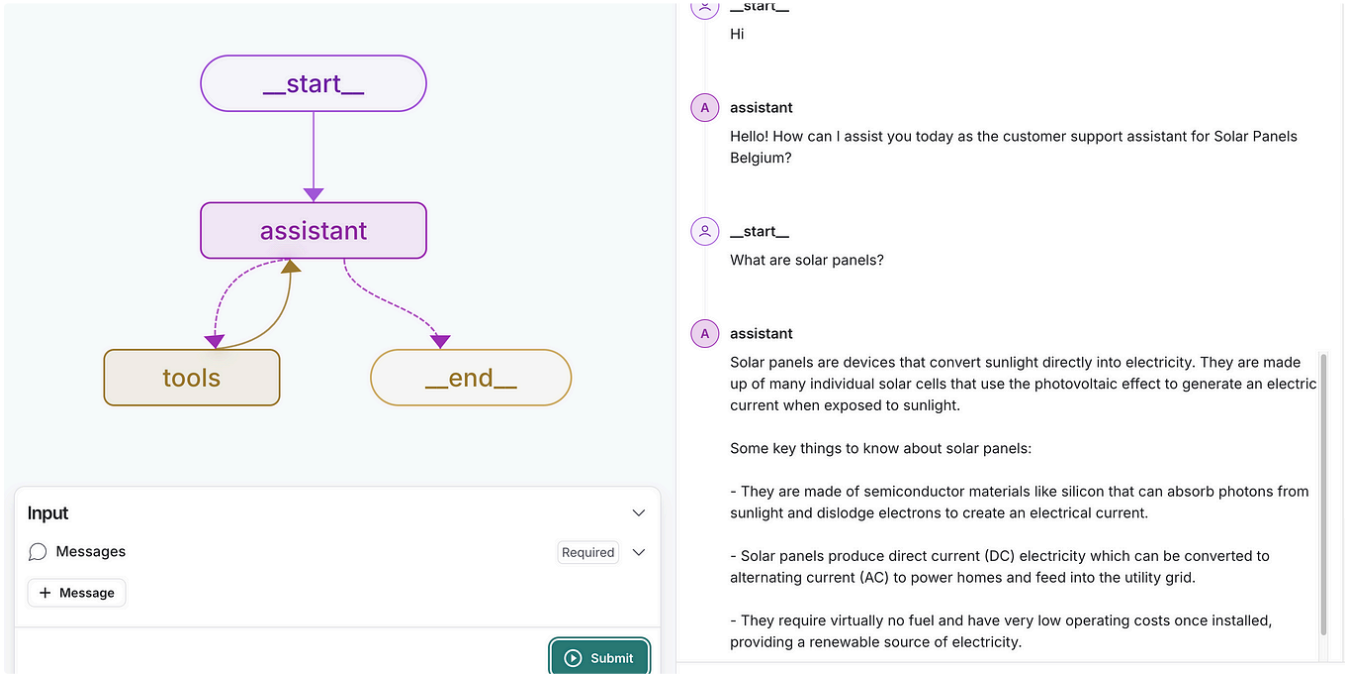


 Chia Jeng Yang in WhyHow.AI

Dynamic Expert Feedback: KG RAG as a Two-Way Retrieval & Memory System vs One-Way Vector System

How up-to-date dynamic context requires feedback and schemas that allow structured communication between humans and LLMs.

Sep 19  60  1



 Lore Van Oudenhove in AI Advances

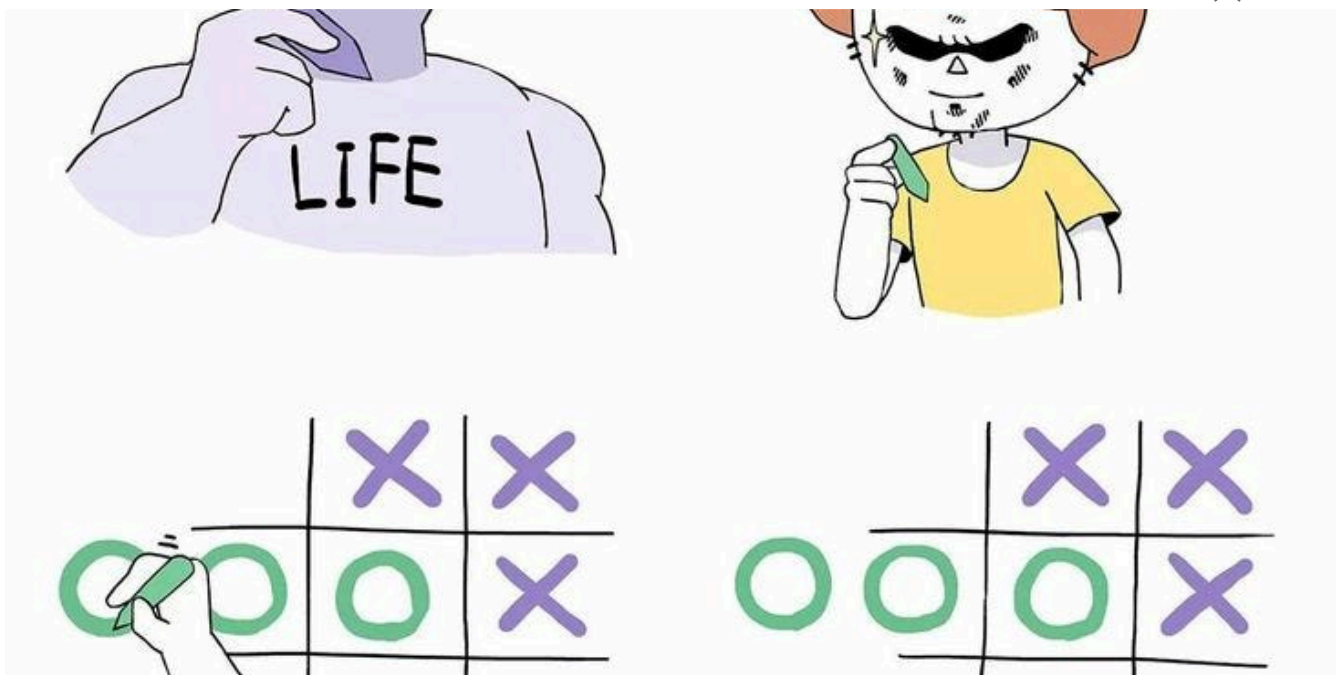
LangGraph Studio: Visualizing and Testing AI Agents with LangChain

A guide to using LangGraph Studio for visualizing, testing, and developing AI agents with LangChain. Includes setup and key features.

6d ago

👍 371

💬 6



Amit Yadav

Langchain vs Haystack

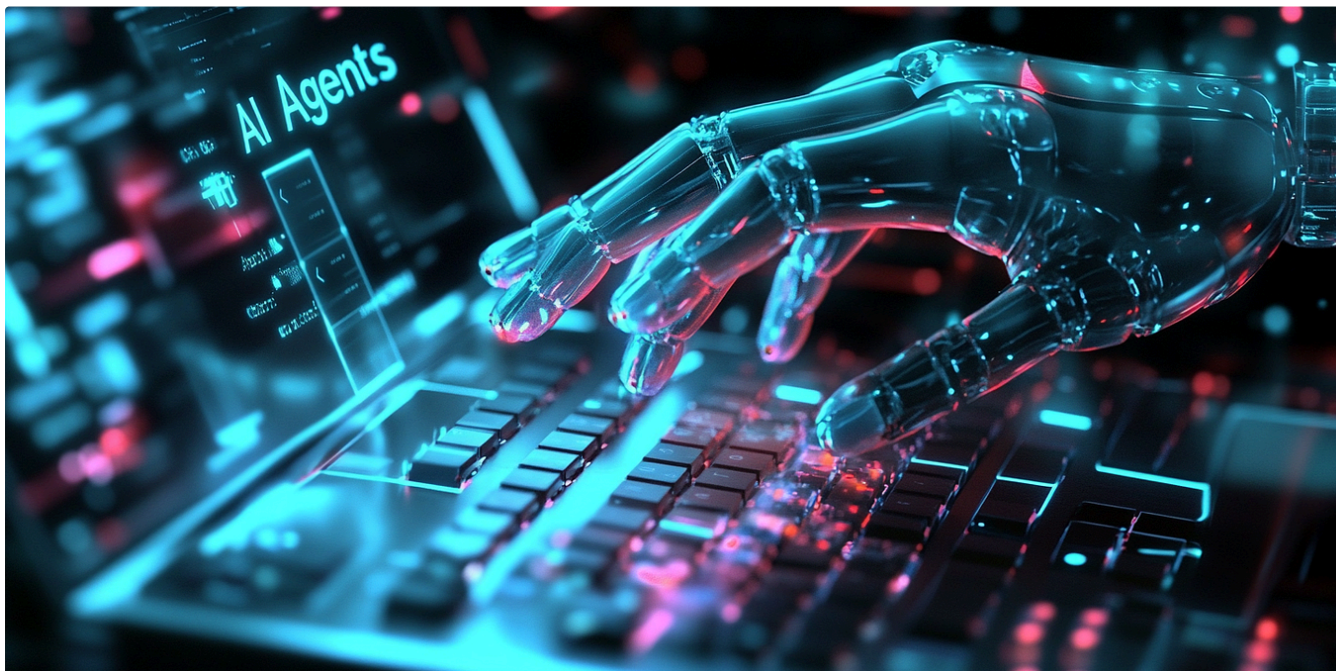
Non-Medium Member? Click here to read the full story.



Aug 6

👍 172

💬 1



Giancarlo Mori

The Shift from Prompt Engineering to Agent Engineering

The launch of ChatGPT in late 2022 marked a significant milestone in the advancement of AI, demonstrating the potential of large language...

★ Sep 9 🖱 102



See more recommendations